



Тест-4-3

Вам не разрешен просмотр этого теста. ✕

|→ **Открыто с:** Friday, 6 March 2026, 22:05 ... **Закрывается:** Saturday, 7 March 2026, 22:05

Разрешено попыток: 1

Ограничение по времени: 10 мин.

Ваши попытки

ПОПЫТКА 1			
Состояние Завершены	Тест начат Saturday, 7 March 2026, 13:34	Завершен Saturday, 7 March 2026, 13:41	Затраченное время 6 мин. 47 сек.
Не разрешается			

У Вас больше нет попыток

Вернуться к курсу

Вопрос 1

Пока нет ответа

Отметить вопрос

Какое преимущество обертывания (wrappers) указано на слайде 19?

Выберите один ответ:

- ☐ a. Увеличивается скорость выполнения программы.
- ☒ b. Не меняется исходный код (работает с чужими/legacy классами).
- ☐ c. Обязательное использование множественного наследования.
- ☐ d. Уменьшается количество классов в программе.

[Очистить мой выбор](#)

Следующая страница

Вопрос 2

Пока нет ответа

Отметить вопрос

Какую проблему решает принцип разделения интерфейсов (Interface Segregation Principle)?

Выберите один ответ:

- ☒ a. Классы вынуждены реализовывать методы, которые им не нужны (как робот, реализующий eat() и sleep()).
- ☐ b. Интерфейсы не поддерживают шаблоны.
- ☐ c. Классы становятся слишком маленькими и их трудно найти.
- ☐ d. Интерфейсы не могут быть унаследованы.

[Очистить мой выбор](#)

[Предыдущая страница](#)

[Следующая страница](#)

Вопрос 3

Пока нет ответа

Отметить вопрос

Как было предложено исправить иерархию классов птиц для соблюдения LSP (слайд 5)?

Выберите один или несколько ответов.

Выберите один или несколько ответов:

- ☐ a. Убрать пингвина из иерархии птиц.
- ☐ b. Сделать метод летать() пустым для пингвина.
- ☒ c. Ввести абстрактный класс Птица с методом двигаться(), и разделить птиц на летающих и нелетающих.
- ☐ d. Оставить всё как есть, но добавить проверку на тип пингвина в функции.

Предыдущая страница

Следующая страница

Вопрос 4

Пока нет ответа

Отметить вопрос

Какой пример на слайде 9 иллюстрирует нарушение принципа инверсии зависимостей (Dependency Inversion Principle)? Выберите один или несколько ответов.

Выберите один или несколько ответов:

- ☐ а. Класс Good использует ссылку на абстрактный класс Database.
- ☐ б. Отсутствие виртуальных функций.
- ☐ в. Шаблонный класс TemplateGood.
- ☒ г. Класс Bad содержит конкретное поле MySQLDatabase db.

Предыдущая страница

Следующая страница

Вопрос 5

Пока нет ответа

Отметить вопрос

Что такое паттерн "Обертывание" (Wrapper) согласно слайду 11?

Выберите один ответ:

- ☐ а. Паттерн для создания одиночек (singleton).
- ☒ б. Паттерн, где один класс "оборачивает" другой, добавляя новую функциональность без изменения исходного класса.
- ☐ в. Паттерн, который позволяет создавать новые классы на основе существующих через наследование.
- ☐ г. Паттерн для работы с базами данных.

[Очистить мой выбор](#)

Предыдущая страница

Следующая страница

Вопрос 6

Пока нет ответа

Отметить вопрос

Какой тип обертки (слайды 13-15) используется для изменения интерфейса класса, чтобы он соответствовал ожиданиям клиента?

Выберите один ответ:

- ☒ a. Адаптер (Adapter).
- ☐ b. Фасад (Facade).
- ☐ c. Декоратор (Decorator).
- ☐ d. Компоновщик (Composite).

[Очистить мой выбор](#)

[Предыдущая страница](#)

[Следующая страница](#)

Вопрос 7

Пока нет ответа

Отметить вопрос

Что означает фраза "классы открыты для расширения, но закрыты для изменения" (Open/Closed Principle)?

Выберите один ответ:

- ☐ a. Класс должен иметь открытые поля для доступа извне.
- ☐ b. Класс должен быть финальным (final) и запрещать наследование.
- ☒ c. Можно добавлять новую функциональность (через наследование или шаблоны), не изменяя уже существующий и протестированный код.
- ☐ d. Класс нельзя изменять даже для исправления ошибок.

[Очистить мой выбор](#)

[Предыдущая страница](#)

[Следующая страница](#)

Вопрос 8

Пока нет ответа

Отметить вопрос

Кем и когда была предложена аббревиатура SOLID?

Выберите один или несколько ответов.

Выберите один или несколько ответов:

- ☒ a. Робертом Мартином в начале 2000-х.
- ☐ b. Линусом Торвальдсом при создании ядра Linux.
- ☐ c. Бьёрном Страуструпом в 1990-х.
- ☐ d. Эриком Гаммой (из "Банды четырёх") в 1994 году.

Предыдущая страница

Следующая страница

Вопрос 9

Пока нет ответа

Отметить вопрос

В первом примере принципа LSP (слайд 4) класс `Пингвин` наследует класс `Птица` с методом `летать()`. Почему этот код нарушает принцип подстановки Лисков (LSP)?

Выберите один или несколько ответов

Выберите один или несколько ответов:

- ☐ a. Потому что метод `летать()` не является виртуальным.
- ☒ b. Потому что пингвин — подтип птицы, но не может летать, что приводит к исключению при использовании через указатель на базовый класс.
- ☐ c. Потому что у пингвина нет метода `ходить()`.
- ☐ d. Потому что класс `Птица` не является абстрактным.

Предыдущая страница

Следующая страница

Вопрос 10

Пока нет ответа

Отметить вопрос

Согласно принципу единственной ответственности (Single Responsibility Principle), что должен делать класс?

Выберите один ответ:

- ☐ а. Класс должен реализовывать как можно больше методов.
- ☐ б. Класс должен зависеть от абстракций, а не от конкретных классов.
- ☐ в. Класс должен быть открыт для расширения, но закрыт для изменения.
- ☒ г. Класс должен делать что-то одно (иметь одну причину для изменения).

[Очистить мой выбор](#)

[Предыдущая страница](#)

[Закончить попытку...](#)